

HEXON

Product, Game and Engineering Specification

Normative specification for implementation, testing, operation and extension

Greenfield edition · Version 1.0 · 5 October 2026

Audience: game engineers, frontend engineers, systems architects, ML engineers, audio programmers, QA, designers, and coding agents.

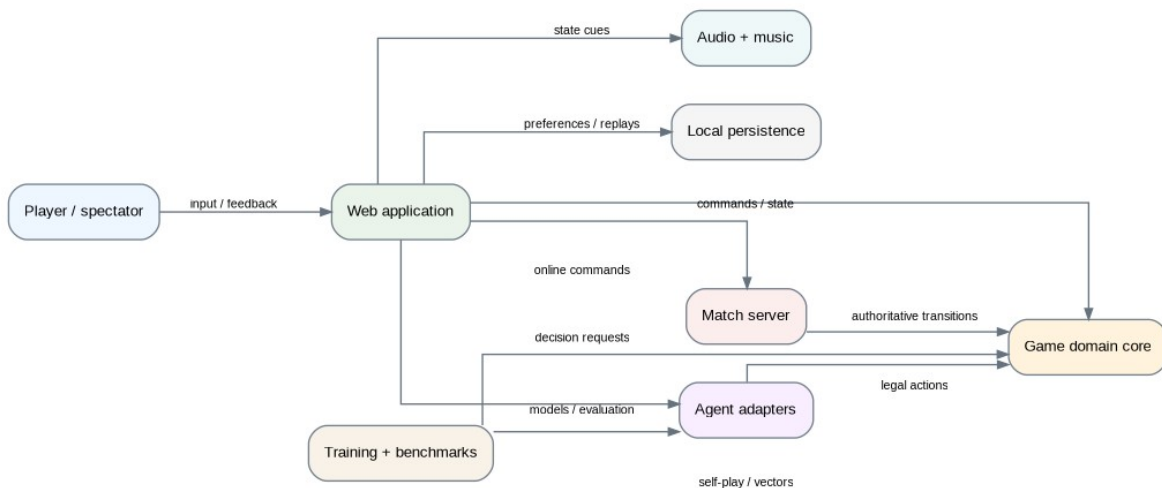


Exhibit 0 — Whole-system context — Every subsystem is a client of one authoritative game domain rather than an alternative definition of the rules.

Normative language

MUST and MUST NOT are mandatory requirements. SHOULD and SHOULD NOT are strong recommendations that may be violated only with an explicit architectural decision record. MAY indicates an optional capability.

0. Contents and reading map [DOC-INDEX]

This document is organized so a human can read it as a product and architecture narrative while a machine can retrieve precise contracts, state definitions, algorithms, examples, and acceptance criteria. Each major section owns one class of truth; duplicated summaries are explanatory, not alternative specifications.

- 1 Product definition and design principles
- 2 Canonical game specification
- 3 Canonical state and data model
- 4 Game-engine architecture and APIs
- 5 Application architecture
- 6 Runtime workflows and concurrency
- 7 GUI and interaction design
- 8 Frontend state architecture
- 9 Player and agent interface
- 10 Classical search and solver
- 11 Neural opponent architecture
- 12 Reinforcement-learning system
- 13 Benchmark and LLM evaluation architecture
- 14 Audio and adaptive soundtrack
- 15 Generative music and musical theory
- 16 Game-to-music / Strudel system
- 17 Online multiplayer architecture
- 18 Persistence and data contracts
- 19 Performance, security and observability
- 20 Accessibility and inclusive design
- 21 Testing strategy
- 22 Build, packaging and deployment
- 23 Recommended package tree and dependency policy
- 24 Configuration registry
- 25 Engineering standards and development workflow
- 26 Acceptance specification
- 27 Reference ontology and glossary
- Appendices — schemas, pseudocode, examples and implementation artifacts

Fast path for implementers

Build in this order: domain core -> golden replay tests -> minimal board UI -> match controller -> human play -> agents -> multiplayer -> audio -> training/benchmarks -> hardening. Never implement a second copy of game legality in a UI, bot, server, or notebook.

1. Product definition and design principles [PROD]

HEXON is a two-player, zero-sum strategy game played on a finite square grid. Players construct a shared board using squares and circles, convert local configurations through triangle actions, and permanently score by creating hexes. Most actions are deterministic; triangle resolution contains one explicit chance event, making the game a stochastic perfect-information game.

The application should feel substantially simpler than its internal architecture. A player sees a board, a small vocabulary of pieces, clear legal actions, immediate visual and auditory feedback, and an opponent. Internally, domain state, orchestration, presentation state, AI computation, networking, persistence, and media generation remain separate so that each can evolve without corrupting the others.

Primary product goals

- Rules that can be explained in minutes but support deep search, tactical conversions, resource constraints, and positional play.
- Fast local play on phones and desktops, with no account required for the core experience.
- Interchangeable human, search, neural, remote, and LLM agents behind one decision interface.
- Deterministic replay of every match, including chance outcomes, so debugging, training, benchmarking, and generative media share one event history.
- Accessible interaction: touch, mouse, keyboard, screen reader, reduced motion, and optional voice input.
- Offline-first local play, with online multiplayer as an optional network capability.
- An adaptive soundtrack and optional game-to-music export that use game state as data rather than embedding game rules in the media subsystem.

Architectural principles

Principle	Requirement
One rules authority	Only the domain package decides legality, state transitions, chance outcomes, scoring, and terminal results.
Pure core	Core rule functions are deterministic given state, action, and an explicit random/chance input; they perform no I/O.
Ports and adapters	UI, networking, storage, agents, audio, and providers communicate through typed contracts.
Replay first	Every accepted command and every resolved chance outcome can be serialized into an event record and replayed.
Immutable boundaries	Application code treats published GameState values as immutable snapshots; mutation, if used internally for performance, never leaks across package boundaries.
Explicit async ownership	Every long-running operation has a request identity and cancellation signal; stale bot, network, and analysis results are discarded.
Progressive enhancement	The board remains playable when advanced hints, voice, music, or online services are unavailable.
Machine-readable contracts	Important concepts have stable IDs, schemas, enums, and examples in addition to prose.

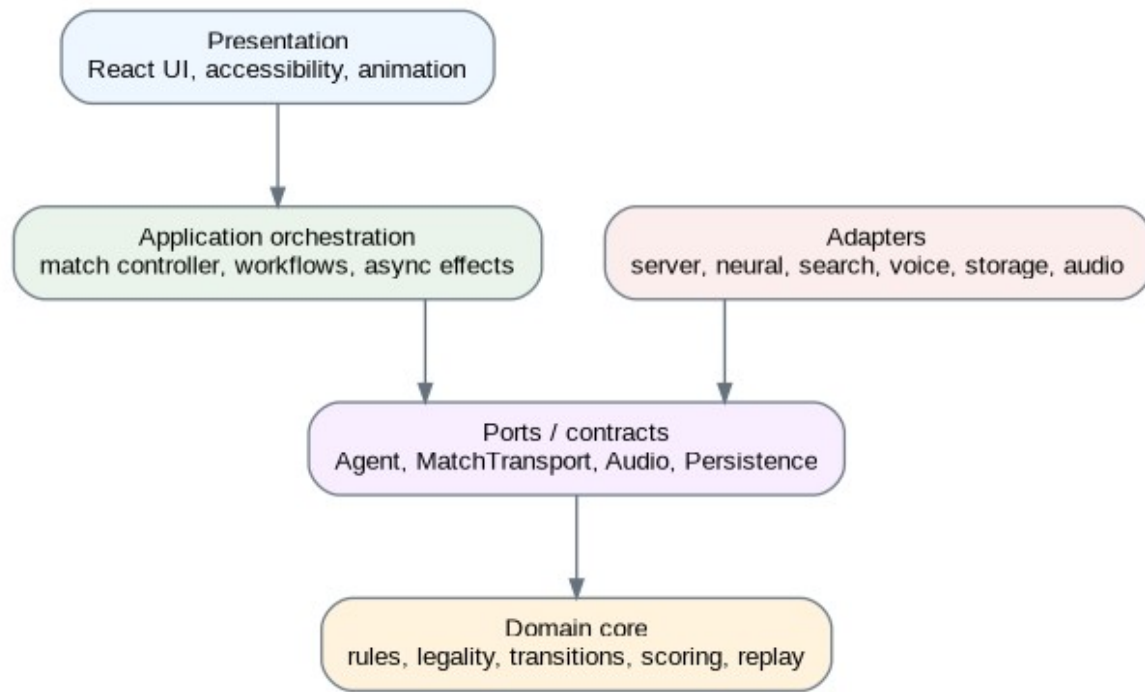


Exhibit 1 — Layered architecture — Dependency direction points toward stable contracts and the domain core. Adapters depend on ports; the domain does not depend on React, networking, storage, or audio.

Non-goals

- No hidden rule changes based on theme, device, opponent, or network mode.
- No server-authoritative UI rendering; clients render from canonical state snapshots.
- No reward shaping that changes the game objective during neural training.
- No audio file dependency for the adaptive soundtrack; synthesis may be generated or pre-rendered locally for performance.
- No AI agent is trusted to produce a legal move; legality is always enforced by the domain.

2. Canonical game specification [GAME]

A match has two players, conventionally Blue (player 0) and Red (player 1), and a square board of size $S \times S$. The default board is 5×5 ; 7×7 is an extended mode. Cells use row-major indexing: $\text{cell} = \text{row} \times S + \text{column}$.

Piece vocabulary and ownership

Piece	Persistence	Placement / effect
Square	Persistent until removed by a triad	May be placed in any empty cell while the mover has square inventory remaining.
Circle	Persistent until removed by a triad	May be placed in an empty cell adjacent, including diagonals, to at least one square of either player.
Triangle	Transient event	May be placed in an empty cell adjacent to at least one valid circle-square pair where at least one member of the pair belongs to the mover. The triangle is removed during resolution.

Hex	Permanent score piece	After a triangle resolves, the same player places one hex in one of the three cleared cells. Hexes are never removed.
Pass	No piece	Legal only when the current player has no legal square, circle, or triangle placement.

Neighborhood

Adjacency uses the Moore neighborhood: the eight orthogonal and diagonal cells surrounding a cell, clipped at board edges. The same adjacency relation is used for circle eligibility, triangle pair construction, and connected-hex groups.

$$N(i) = \{ j \mid \max(|\text{row}(j) - \text{row}(i)|, |\text{col}(j) - \text{col}(i)|) = 1 \}$$

Square inventory

Each player begins with a finite square inventory. The required default is $\text{round}(S^2/3)$: 8 squares on a 5×5 board and 16 on a 7×7 board. A removed square does not return to inventory. The remaining inventory is therefore authoritative state and cannot be inferred from the board picture.

Triangle resolution and chance

For a triangle placed at cell t by player p , construct all adjacent pairs (c,s) such that c contains a circle, s contains a square, and $\text{owner}(c)=p$ or $\text{owner}(s)=p$. If the set is empty, the action is illegal. Otherwise choose exactly one pair uniformly at random. Remove the triangle, chosen circle, and chosen square; these three cells become the legal destinations for the mover's hex.

$$\begin{aligned} \text{Pairs}(t,p) &= \{(c,s) \mid c,s \in N(t), \text{piece}(c)=\text{CIRCLE}, \text{piece}(s)=\text{SQUARE}, \text{owner}(c)=p \vee \text{owner}(s)=p\} \\ P((c,s) \text{ chosen}) &= 1 / |\text{Pairs}(t,p)| \end{aligned}$$

Turn semantics

A square or circle ends the turn immediately. A triangle does not: it enters a mandatory pending-hex substate controlled by the same player. Forced pass ends the turn. Turn ownership alternates after a completed turn, not after every atomic placement.

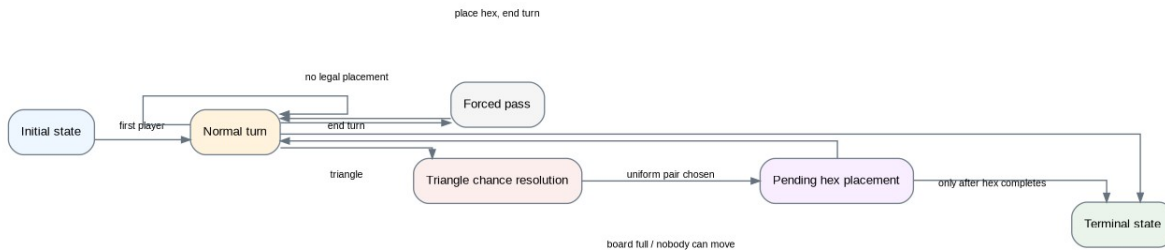


Exhibit 2 — Domain state machine — The pending-hex state is part of the same player turn. Terminal evaluation is suppressed until the pending hex has been placed.

Game end and scoring

The game ends when the board is full, or when neither player has any legal placement. A pending hex must always be completed before terminal evaluation. The winner is the player with more hexes. If hex counts tie, compare the size of each player's largest 8-connected hex group. If those also tie, the result is a draw.

$$\begin{aligned} \text{score_primary}(p) &= \text{number of HEX cells owned by } p \\ \text{score_tiebreak}(p) &= \text{max size of an 8-connected component of } p\text{-owned HEX cells} \end{aligned}$$

Opening player

A fair match chooses the first player uniformly at random. The random decision is part of match metadata so it can be replayed exactly. Tests and deterministic experiments MAY supply a fixed starting player explicitly.

Worked move example

Example: T12-16-17-16 on 5×5

The mover places a triangle at cell 12. A valid adjacent circle at 16 and square at 17 are selected by the chance resolver. Cells 12, 16, and 17 are cleared. The same mover then places a permanent hex at cell 16. Only after that hex placement does the turn end.

3. Canonical state and data model [STATE]

The canonical state contains only information required to determine future legal actions and outcomes, plus a small set of explicit replay/UI markers. Derived values such as legal moves, largest groups, board fill, and winner text are computed, not stored.

Core TypeScript model

```
type PlayerId = '0' | '1';
type Piece = 'EMPTY' | 'SQUARE' | 'CIRCLE' | 'HEX';
interface Cell { piece: Piece; owner: PlayerId | null }
interface GameState {
  version: 1; size: 5 | 7; board: readonly Cell[];
  playerToMove: PlayerId; squaresLeft: Record<PlayerId, number>;
  hexCount: Record<PlayerId, number>; pendingHex: readonly number[] | null;
  turnNumber: number; result: GameResult | null;
}
interface ChanceResolution { triangle: number; circle: number; square: number }
```

State invariants

ID	Invariant
INV-01	<code>board.length = size².</code>
INV-02	EMPTY cells have owner=null; non-empty cells have a valid player owner.
INV-03	$0 \leq \text{squaresLeft}[p] \leq \text{round}(\text{size}^2/3).$
INV-04	<code>hexCount[p]</code> equals the number of HEX cells owned by p.
INV-05	<code>pendingHex</code> is either null or exactly three distinct empty cells produced by one resolved triangle.
INV-06	When <code>pendingHex</code> $\neq$ null, only a hex placement by <code>playerToMove</code> is legal.
INV-07	A terminal result is present only when <code>pendingHex</code> = null.
INV-08	Replaying the event log from the initial state reproduces the same canonical state hash.

Action model

```
type Action =
  | { kind: 'PLACE_SQUARE'; cell: number }
  | { kind: 'PLACE_CIRCLE'; cell: number }
  | { kind: 'PLACE_TRIANGLE'; cell: number }
  | { kind: 'PLACE_HEX'; cell: number }
  | { kind: 'PASS' };
```

For neural and solver interfaces, actions also have a compact integer encoding: $a = \text{kind} \times S^2 + \text{cell}$, with kind 0=square, 1=circle, 2=triangle, 3=hex; pass = $4 \times S^2$. This yields 101 actions on 5×5 and 197 on 7×7 .

Event and replay model

```
type GameEvent =
  | { type: 'MATCH_STARTED'; size: number; firstPlayer: PlayerId; seed: string }
  | { type: 'ACTION_ACCEPTED'; player: PlayerId; action: Action }
  | { type: 'TRIAD_RESOLVED'; player: PlayerId; triangle: number; circle: number; square: number }
  | { type: 'MATCH_ENDED'; result: GameResult };
```

The replay stream records the resolved chance outcome explicitly. A replay therefore never depends on rerunning a random generator. The random seed is preserved for experiments and diagnostics, but the event history is sufficient for deterministic reconstruction.

Data dictionary

Field	Type	Meaning	Authoritative?
size	5 7	Board width and height.	Yes
board	Cell[size ²]	Piece and owner per cell.	Yes
playerToMove	PlayerId	Controller of the next decision.	Yes
squaresLeft	Record<PlayerId,int>	Unplayed square inventory.	Yes
hexCount	Record<PlayerId,int>	Primary score; validated against board.	Yes
pendingHex	number[3] null	Legal hex cells after resolved triangle.	Yes
turnNumber	int	Completed-turn counter for replay/UI.	Yes
legalActions	Action[]	Computed from state.	No
largestGroup	int	Computed tiebreak statistic.	No
boardFill	0..1	Computed density used by UI/music.	No

4. Game-engine architecture and APIs [CORE]

The game engine is a pure TypeScript package with no React, DOM, storage, audio, transport, or model-runtime imports. Its public API is intentionally small: create state, enumerate legal actions, apply one action with an explicit chance source, compute result, serialize/replay, and expose geometry helpers.

Recommended public API

```
createInitialState(config): GameState
legalActions(state): readonly Action[]
validateAction(state, action): ValidationResult
applyAction(state, action, rng): TransitionResult
computeResult(state): GameResult | null
replay(events): GameState
hashState(state): string
encodeAction(action, size): number
decodeAction(index, size): Action
```

Transition result

```
interface TransitionResult {
  state: GameState;
  events: readonly DomainEvent[];
  chance?: ChanceResolution;
}
```

The transition function returns domain events such as `PIECE_PLACED`, `TRIAD_RESOLVED`, `HEX_SCORED`, `TURN_ENDED`, and `MATCH_ENDED`. Presentation and audio consume these events, but the rules engine does not know what animation or sound will be played.

Transition pseudocode

```
applyAction(state, action, rng):
  assert !state.result
  if state.pendingHex:
    require action.kind == PLACE_HEX and cell in pendingHex
    place permanent hex; increment score; clear pendingHex; endTurn()
  else if action is PLACE_SQUARE:
    require empty && squaresLeft>0; place; decrement inventory; endTurn()
  else if action is PLACE_CIRCLE:
    require empty && adjacentSquareExists(); place; endTurn()
  else if action is PLACE_TRIANGLE:
    pairs = triadPairs(state, cell, playerToMove); require pairs not empty
    pair = uniformChoice(pairs, rng); clear triangle/circle/square; pendingHex = those 3 cells
  else if action is PASS:
    require no legal placement; endTurn()
  if turn ended and pendingHex is null: state.result = computeResult(state)
  return state + domain events
```

Determinism boundary

Rule

The only nondeterministic rule operation is selection of the circle-square pair during a triangle. Randomness enters through an injected `RandomSource` or through an explicit resolved pair. No rule function may call `Math.random()`, `Date.now()`, the network, or browser state.

Geometry and complexity

Precompute each cell's 8-neighbor list for each supported board size. With $S \leq 7$, straightforward scans are already cheap; prioritize correctness and stable semantics over micro-optimization. Use bitsets or typed arrays only inside search/neural packages where millions of transitions justify them.

5. Application architecture [ARCH]

The application layer coordinates workflows but does not decide rules. It receives user or agent intents, invokes the domain, publishes the new state, starts or cancels asynchronous work, and translates domain events into adapter calls.

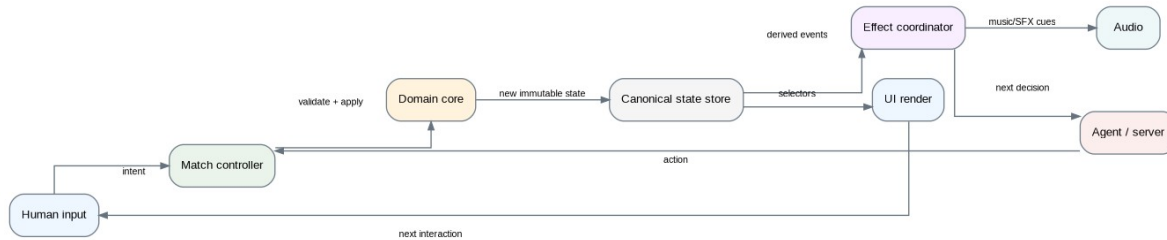


Exhibit 3 — System communication flow — All feedback loops return through the match controller; long-running work is an effect, never hidden inside a React component.

Subsystem responsibilities

Subsystem	Owns	Must not own
game-core	Rules, state transitions, scoring, replay grammar	UI, network, audio, timers
match-controller	Session lifecycle, player assignment, async sequencing, cancellation	Rule duplication
ui	Rendering, interaction, accessibility, animation state	Legality or winner calculation
agents	Decision policy over legal actions	Direct state mutation
match-server	Seat assignment, authoritative command ordering, snapshots	Presentation
audio	SFX/music scheduling and DSP	Rule interpretation beyond supplied cues
persistence	Settings, replays, schema migration	Live game authority
training/benchmarks	Data generation, optimization, evaluation	Alternative rule engine

Dependency rule

```

apps/* -> application -> ports -> domain
adapters/* -> ports
training/* -> domain + agent/search packages
NO dependency may point from domain to adapters or apps
  
```

6. Runtime workflows and concurrency [FLOW]

Application bootstrap

1. Load immutable build metadata and capability flags.
2. Load and validate persisted settings; fall back field-by-field to defaults.
3. Initialize the presentation shell and board-independent services.
4. Lazily initialize audio only after a user gesture.
5. Lazily load neural/search workers only when the selected mode requires them.
6. When entering the board screen, create or join a MatchSession and render from its canonical snapshot.

Human move workflow

```
pointer/keyboard/voice intent -> UI command -> MatchController.submit(action) -> game-core.validate -> game-core.apply ->
publish snapshot -> animate + sound domain events -> request next actor decision
```

Bot move workflow

A bot decision request captures matchId, stateHash, playerToMove, legal actions, difficulty parameters, and an AbortSignal. When the result returns, the controller verifies that the request still matches the live session and state hash before submitting the action. A stale result is silently discarded.

Triangle workflow

```
PLACE_TRIANGLE
-> core enumerates valid pairs
-> authoritative RNG selects one pair
-> TRIAD_RESOLVED event clears 3 cells
-> state.pendingHex=[triangle,circle,square]
-> same player chooses PLACE_HEX
-> core scores hex and ends turn
```

New game / cancellation

Starting a new game increments a session generation. Every worker request, network subscription, animation queue, voice recognition session, and delayed effect is associated with that generation and cancelled or ignored when the generation changes.

Failure containment

Failure	Required behavior
Neural model fails to load	Fall back to deterministic heuristic/search agent or disable that opponent; local human play remains available.
Worker blocked/unavailable	Run analysis in bounded main-thread slices with cancellation, or disable hints without affecting play.
Audio context denied	Continue silently; never block game start.
Online server unreachable	Offer offline human play and bots; preserve selected settings.
Speech recognition unavailable	Hide/disable voice control for the session; pointer, keyboard, and touch remain complete.
Persisted settings corrupt	Validate with schema, preserve valid fields, reset invalid fields only.

7. GUI and interaction design [GUI]

The interface is intentionally low-friction: setup, board, help, and result are the only primary surfaces. The visual identity may vary through piece themes, but themes are skins over stable semantics; they never alter rules, agents, or music logic.

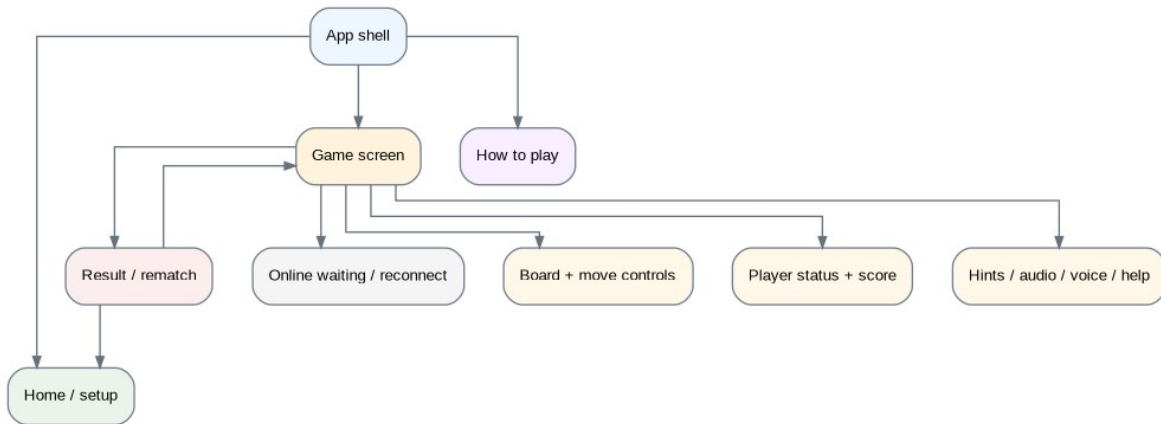


Exhibit 4 — Screen and component tree — The board screen is the operational center; help and results are overlays or adjacent states rather than independent rule contexts.

Home / setup screen

The setup screen **MUST** expose board size, opponent, difficulty/mode where relevant, local/online human mode, piece theme, hints, in-game messages, sound, music, and optional voice capability. Changes apply to the next match. Settings are persisted locally after validation.

Control	Behavior
Board	5×5 default; 7×7 advanced.
Opponent	Human, ne0n, xen0n; additional adapters MAY appear if installed.
Mode	Normal favors variety; Turbo lowers sampling temperature and plays closer to the top-rated action. Hidden for human-vs-human.
Human mode	Online matchmaking or offline same-device.
Theme	Cycles visual piece skins; semantic names remain available to assistive technology.
Hints	Shows legal cells and permits optional deeper engine suggestions.
Messages	Controls nonessential narrative/toast copy, not accessibility announcements.
Sound / music	Independent toggles.
Voice	Visible capability setting when browser support exists; never hidden behind an easter egg.

Board screen

The board occupies the visual center. Each cell is a semantic button/gridcell with a stable accessible label containing coordinate, piece, owner, and whether the cell is legal for the selected action. Selecting a piece type

highlights legal destinations. When a triangle resolves, only the three pending hex destinations remain interactive.

Player status panels

For each player show: active-turn state, squares remaining, hex score, and largest connected hex group. Do not encode active player or winner by color alone. The same values should be available in accessible text.

Result presentation

At terminal state, freeze rule input immediately, announce the result, and present New game / Rematch / Settings actions. A short celebratory animation MAY run, but reduced-motion mode replaces flicker with steady emphasis. The result is derived from canonical `GameResult`, never recomputed by the UI.

Tutorial

The tutorial is an interactive, deterministic micro-match on a reduced board. It teaches goal, board indexing, the four piece semantics, forced hex placement, square inventory, scoring, tiebreak, and pass. User-controlled steps never auto-advance past an expected interaction. The tutorial uses the same piece renderer and terminology as the main board but a scripted state provider rather than the live match controller.

Responsive layout

Viewport	Layout
Phone portrait	Single-column board-first layout; large touch targets; status compressed above/below board.
Phone landscape	Prefer a responsive two-column layout; if the board cannot remain usable, show a non-destructive orientation advisory rather than pausing the match.
Tablet	Board plus persistent status/tools rail when space allows.
Desktop	Centered board with status and controls in adjacent columns; no excessive stretching.
Short window	Board keeps a minimum practical size and the screen scrolls vertically.

Theme system

Ship seven visual skins: geometric shapes, ponies, chess, cyber, fantasy, music, and clothing. A theme defines four SVG outlines, a scoring-piece inner mark where needed, display nouns, spoken aliases, and colors. The semantic piece identity remains `SQUARE/CIRCLE/TRIANGLE/HEX` regardless of theme.

Interaction and accessibility

- Keyboard is fully supported: Tab/Shift+Tab moves through controls, arrow keys navigate the board grid, Enter/Space activates a selected cell, Escape closes dismissible overlays.
- Pointer and touch targets are at least about 44 CSS px where practical.
- Focus is never lost when overlays open/close; focus returns to the invoking control.
- Animations obey prefers-reduced-motion.
- The board uses non-color cues for ownership and legality.
- Voice input is optional and additive; every voice action has a complete manual equivalent.

8. Frontend state architecture [UI-STATE]

Separate canonical domain state from presentation state. React components should consume selectors and dispatch intents; they should not hold independent copies of rules-critical data.

Concurrent state machines

Machine	Representative states
App navigation	setup -> game -> result; help overlay orthogonal
Match lifecycle	creating -> active -> terminal -> disposed
Turn interaction	choose-piece -> choose-cell -> awaiting-resolution -> pending-hex
Bot request	idle -> requested -> thinking -> resolved/cancelled/error
Online	offline matchmaking -> waiting -> connecting -> playing -> reconnecting -> ended
Audio	locked -> ready -> muted/unmuted -> suspended
Voice	unsupported idle -> requesting-permission -> listening -> error
Hints	disabled idle -> analysing -> report-ready

Side-effect ownership

Use effect services for workers, timers, storage, audio, speech, and transport. Components may request effects but never create unmanaged long-lived resources. Every effect returns a cleanup function or obeys AbortSignal.

Recommended frontend packages

Concern	Recommended package	Rationale
UI	React + TypeScript	Component model and strong typing.
Build	Vite	Fast dev server, worker support, deterministic production bundles.
Styling	Tailwind CSS plus CSS custom properties	Utility layout with theme tokens; avoid runtime CSS-in-JS overhead.
Workflow state	XState	Explicit concurrent state machines for match, online, voice, hints, and navigation.
Schema validation	Zod	Runtime validation of storage, network messages, model metadata, and replay files.
Testing	Vitest + React Testing Library + Playwright	Unit/component/browser layers.

9. Player and agent interface [AGENT]

Every non-authoritative decision maker implements the same PlayerAgent port. This allows local humans, heuristics, search, neural policies, remote players, and LLMs to be scheduled by the same match controller.

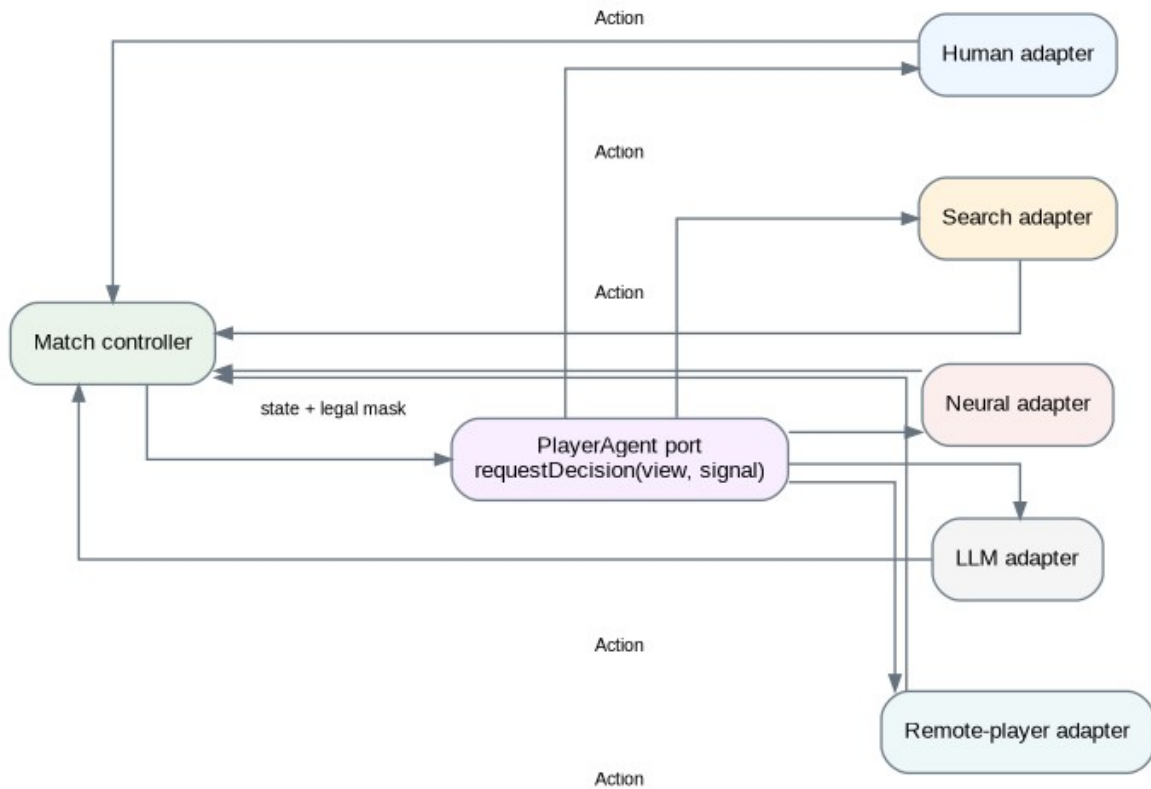


Exhibit 5 — Interchangeable agent contract — Agents receive a read-only decision view and legal actions; they return an action proposal. The controller and domain remain authoritative.

```
interface DecisionRequest {
  matchId: string; stateHash: string; player: PlayerId;
  state: Readonly<GameState>; legalActions: readonly Action[];
  mode: 'normal' | 'turbo'; signal: AbortSignal;
}
interface PlayerAgent {
  id: string; capabilities: AgentCapabilities;
  decide(req: DecisionRequest): Promise<Action>;
}
```

Sampling modes

Agents that expose scores or logits MAY sample instead of always taking argmax. Use softmax temperature T:

$$P(a) = \exp(z_a / T) / \sum_b \exp(z_b / T)$$

T → 0 approaches argmax; higher T increases diversity.

Prescribed defaults for game-facing neural/heuristic opponents: Normal is deliberately more varied; Turbo is closer to best play. Exact temperatures belong in the configuration registry and must be measured per agent because logit scales differ.

Legality firewall

Mandatory

The match controller must reject any agent output not present in legalActions. Agents are advisory components; they cannot mutate state, resolve chance, end a turn, or declare a winner.

10. Classical search and solver [SEARCH]

The solver is a separate high-performance representation of the same game contracts. It may use typed arrays, packed keys, symmetry reduction, caches, and selective modes, but its transitions are verified against golden domain vectors.

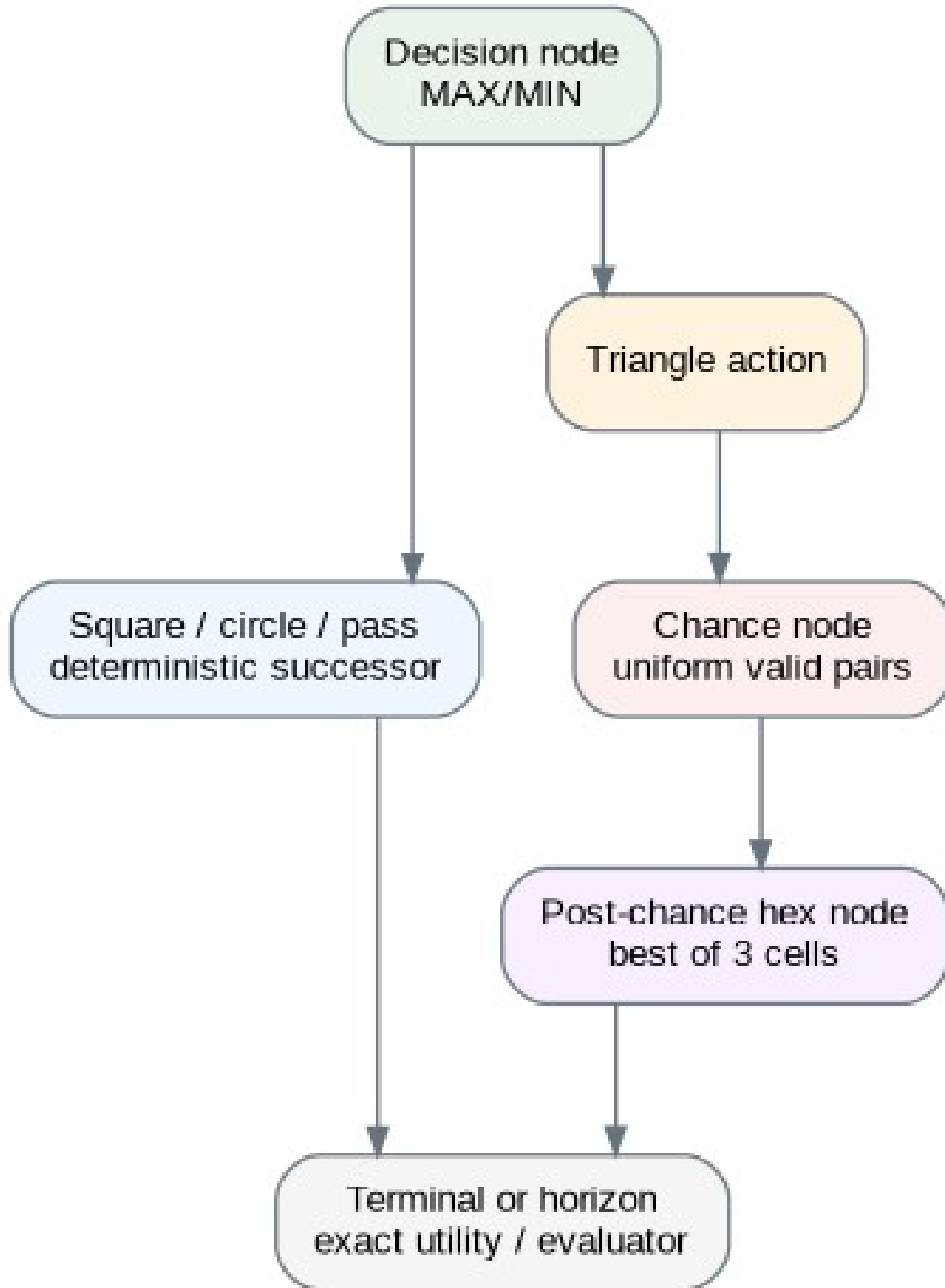


Exhibit 6 — Search tree semantics — Triangle moves introduce a chance node followed by a same-player hex choice. This is expectiminimax, not ordinary alternating minimax.

Expectiminimax

$$\begin{aligned} V(s) &= \max_a V(T(s,a)) && \text{if player 0 moves} \\ V(s) &= \min_a V(T(s,a)) && \text{if player 1 moves} \\ V(\text{chance}) &= \sum_o P(o) V(o) && \text{for triangle pair outcomes} \\ V(\text{hex-node}) &= \max/\min_h V(h) && \text{for the same mover} \end{aligned}$$

Exact reductions

- Alpha-beta pruning or principal-variation search on deterministic player nodes.
- Transposition table keyed by canonical state and search depth/bound type.
- D4 rotations/reflections map symmetric board states to one canonical key.
- State-stabilizer move pruning removes actions equivalent under symmetries of the current position.
- Equivalent-successor merging avoids evaluating duplicate canonical states reached through different local choices.
- Chance-node bounds use the known utility interval $[-1,+1]$ to prove some remaining outcomes irrelevant.

Horizon evaluator

A practical nonterminal evaluator combines primary score, connected-group structure, immediate triangle opportunities, mobility, and square inventory, then caps heuristic values strictly inside terminal utility.

$$\begin{aligned} x &= 2.3 \Delta_{\text{hex}} + 0.28 \Delta_{\text{largestGroup}} + 0.18 \Delta_{\text{triadSpots}} + 0.025 \Delta_{\text{mobility}} + 0.035 \Delta_{\text{squaresLeft}} \\ H(s) &= 0.95 \tanh(x / 3) \end{aligned}$$

Quiescence and selective search

At nominal depth zero, an optional triangle-only quiescence extension resolves immediate conversion tactics before evaluating. A beam mode MAY truncate low-ranked player alternatives for exploratory deep analysis, but it must never drop chance outcomes; it is explicitly approximate and must not be labeled exact.

11. Neural opponent architecture [NN]

Use a compact fully convolutional residual network so the same weights can evaluate 5×5 and 7×7 boards. Inputs are mover-relative, allowing one model to play both colors.

Observation tensor

Plane	Content
0	My squares
1	My circles
2	My triangles (reserved compatibility plane)
3	My hexes
4	Opponent squares
5	Opponent circles
6	Opponent triangles (reserved compatibility plane)
7	Opponent hexes
8	Empty cells
9	Pending-hex legal cells

10	Must place a hex now
11	My squares left / initial supply
12	Opponent squares left / initial supply
13	Constant ones / border reference

Shape is (14,S,S). A legal-action mask of length $4S^2+1$ accompanies the observation.

Reference network

Stage	Operation	Output
Stem	3×3 conv, 14 -> 64, padding 1, ReLU	64×S×S
Residual blocks ×4	3×3 conv -> ReLU -> 3×3 conv -> skip add -> ReLU	64×S×S
Pooling	Spatial mean and spatial max	64 + 64
Policy cells	1×1 conv, 64 -> 4, flatten by piece kind	4S ² logits
Policy pass	Linear on mean pool	1 logit
Value	Linear 128 -> ReLU -> linear 3 on mean+max	win/draw/loss logits
Auxiliary (training only)	Linear 128 -> ReLU -> linear 6	final hexes, groups, squares for both players

For S=5 this reference network has approximately 338k trainable parameters including the auxiliary head. Browser inference exports only the trunk, policy, and value heads.

Value conversion

```
p = softmax([l_win, l_draw, l_loss])
value = p_win - p_loss ∈ [-1,1]
```

Browser runtime

Implement inference in a dedicated worker using Float32Array and explicitly versioned weight metadata. Avoid a large runtime dependency if the network operators are limited to conv/ReLU/linear/pooling; a small verified kernel makes model packaging, offline play, and cross-version parity easier. Maintain golden tests comparing browser inference against training-framework outputs.

12. Reinforcement-learning system [RL]

Train the neural agent through self-play with search in the Gumbel AlphaZero family. The network supplies priors and values; search produces a stronger policy target. No human games and no handcrafted reward shaping are required.

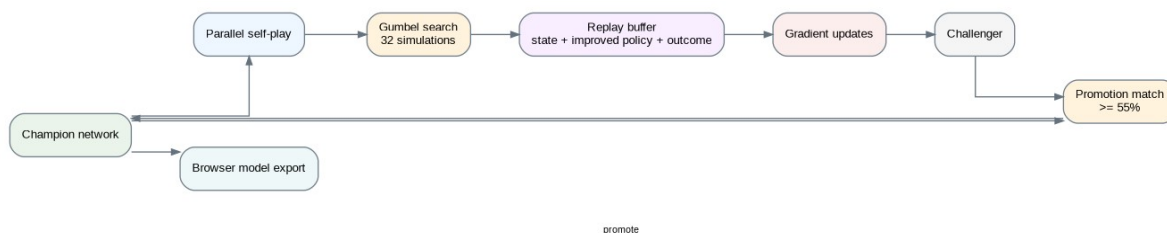


Exhibit 7 — Self-play and promotion pipeline — Only a challenger that clears the promotion gate replaces the champion used for deployment and subsequent evaluation.

Self-play search

Use 32 simulations per decision as the default budget. At the root, add one independent Gumbel sample $g = -\ln(-\ln u)$ to each legal action logit, select a candidate set, and allocate simulations with sequential halving. Below the root, choose actions to reduce the gap between search visits and the improved policy.

Targets

Target	Definition
Policy	Improved distribution returned by search, not raw visit counts.
Value	Final win/draw/loss from the mover's perspective; no intermediate reward shaping.
Auxiliary	Final hex count, largest connected group, and remaining squares for both players. These add learning signal but do not change reward.

Loss function

```

L = λπ Lpolicy + λv Lvalue + Σk λk Laux,k + λ2 ||θ||2
Lpolicy = -Σa πtarget(a) log πθ(a | s)
Lvalue = -Σc yc log pθ(c | s)
Laux = mean squared error over normalized auxiliary targets

```

Replay and augmentation

Store recent decisions in a replay buffer rather than complete serialized games. For each training sample, independently apply one of the board's eight D4 symmetries to observation and action-indexed targets. The pass action is invariant. The exact same permutation must transform every spatial plane and every per-cell action block.

Promotion

Periodically evaluate the challenger against the current champion with balanced seats and fixed evaluation settings. Promote only if the challenger scores at least 55% where draw counts as half a point. Store generation, training seed, hyperparameters, code revision, and evaluation record with every promoted model.

13. Benchmark and LLM evaluation architecture [EVAL]

Evaluation uses the same domain core and PlayerAgent contract as the product. The benchmark harness schedules matches, balances seats, records every decision, and writes resumable append-only result files. Provider-specific code belongs in adapters.

Match record

```

interface MatchRecord {
  runId: string; gameId: string; boardSize: number; seed: string;
  blueAgent: AgentDescriptor; redAgent: AgentDescriptor; firstPlayer: PlayerId;
  events: GameEvent[]; result: GameResult; timingsMs: number[];
  tokenUsage?: Usage; estimatedCost?: number;
}

```

LLM decision protocol

An LLM receives a compact state description, legal actions, coordinate convention, and response schema. The adapter parses the response, validates it against legalActions, retries only for transport/schema failures according to policy, and records the complete prompt/response transcript for reproducibility. Repeated illegal outputs ultimately forfeit; the LLM never receives authority to alter game state.

Statistics

Report wins, draws, losses, score = wins + 0.5×draws, seat-split performance, average move latency, and confidence intervals. Use Wilson intervals for proportions. Elo estimates are secondary summaries and should not substitute for raw match counts and uncertainty.

14. Audio and adaptive soundtrack [AUDIO]

Audio is an adapter driven by domain/application cues. Sound effects correspond to discrete interaction events; the soundtrack responds primarily to slowly varying game state so simultaneous events do not fight for musical control.

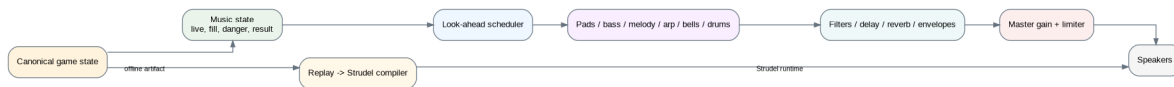


Exhibit 8 — Audio and game-to-music architecture — Live synthesis and offline Strudel generation share the game state as input but have separate rendering paths.

Web Audio graph

Use one AudioContext for music and effects, one master limiter, and explicit gain buses. Notes are scheduled against the audio clock with a look-ahead scheduler; JavaScript timers only wake the scheduler and never define rhythmic precision.

```

step_seconds = 60 / bpm / 4 // sixteenth-note grid
f(midi) = 440 × 2^((midi-69)/12)
    
```

Voice synthesis

A generic pitched voice is oscillator(s) -> low-pass filter -> gain envelope -> dry/reverb/delay buses. Per-note options include waveform, detune, initial/final filter cutoff, attack, release, sustain mode, and effect sends. Chords may share one filter/envelope when that preserves the intended sound and reduces node count.

Drums and space

- Kick/frame drum: sine oscillator with rapid downward pitch sweep and amplitude decay.
- Hi-hat/shaker/brush: short high-passed noise bursts; pre-render multiple takes to avoid unstable phone playback.
- Timpani: pitched kick-like drum tuned to the current harmonic root.
- Reverb: generated stereo impulse response, e.g. decaying and progressively darkened noise.
- Echo: tempo-locked dotted-eighth delay with feedback and a low-pass filter in the feedback loop.
- Master limiter: prevents music and effects from clipping when they overlap.

Mobile strategy

Desktop-class devices may synthesize notes live. Touch/mobile devices MAY render deterministic loop layers through OfflineAudioContext and mix those buffers during play. The musical seed and scheduler logic remain identical so live and pre-rendered paths are perceptually equivalent.

15. Generative music and musical theory [MUSIC]

All soundtrack styles use one engine with configurable tempo, mode, chord-transition table, timbre, and six layers: pad, bass, melody, arpeggio, bells, and drums. Game state controls density and orchestration, not the fundamental rule set.

Harmony

Represent harmony as a weighted directed graph of chord states. The next chord is sampled only from the current chord's permitted successors; repeated entries implement weights. Voicings are hand-selected in close position so successive chords move by small intervals, reducing objectionable voice-leading jumps.

Melody

Maintain a 16-slot scale-degree phrase. Initialize it by a constrained random walk of ± 1 or ± 2 scale steps. Each slot has a persistent rank $r \in [0,1]$; the slot sounds when r is below an intensity-dependent density threshold. Reserve several low-rank anchor positions so the listener perceives a recurring hook. At each chord change mutate only a small number of slots rather than replacing the phrase.

$\text{density}(i) \approx 0.14 + 0.60 i$ for active play, with i = board fill in $[0,1]$
 slot sounds iff $\text{rank}(\text{slot}) < \text{density}(i)$

Game-state mapping

Scene	Intensity	Musical behavior
Quiet / setup	≈ 0.08	Pad, sparse bass/melody/bells; no drums; no arpeggio.
Active game	board fill 0..1	Melody density rises; arpeggio and drums enter at style thresholds; filters may brighten.
One square left	$\min(1, \text{boardFill} + 0.25)$	Force urgency layers; bass may pulse each beat like a heartbeat.
Game over	N/A	Fade active scheduler, play outcome chord under result SFX, then transition to next style.

Seven prescribed styles

Style	Theory / orchestration specification
shapes	C major/minimalist mallet palette. Mostly sine waves, short melodic partial two octaves above, open filter, woodblock-like sine pings. Clear consonance mirrors geometric simplicity.
ponies	C major music-box / nursery character at ~ 108 bpm with light swing. High register, octave partial, root-fifth bouncing bass, frequent bells; the groove borrows from polka and children's-song accompaniment.
chess	A-minor court/early-music color around 66 bpm. Plain triads rather than extended chords, hollow square-wave organ pad, bright decaying sawtooth harpsichord, tuned timpani, reverb without rhythmic echo.
cyber	A-minor synthwave at ~ 76 bpm. Detuned saw pad, intensity-driven filter sweeps, dotted-eighth echo, square arpeggio, electronic hats and kick. Brightness increases with game pressure.
fantasy	D Dorian around 60 bpm. Minor tonic with raised sixth gives a

	modal, hopeful color; slow sine flute attack, wide pad detune, lingering harp-like arpeggio, uneven frame-drum pattern and frequent bells.
music	Calm C-major chamber language around 80 bpm. Filtered triangle waves approximate soft strings/piano, slow harp-like arpeggio, sparse low drum and distant chime.
clothing	Lounge jazz around 104 bpm with strong swing. Faster harmonic rhythm, forced ii-V-I motion, rootless extended voicings, syncopated comping, walking bass, brushes and kick.

Outcome harmony

Outcome	Suggested sonority	Reason
Win	C major add9 spread across registers	Bright, stable, open consonance.
Loss	Low Dm7 with dark filtering	Minor color and reduced high-frequency energy.
Draw	A suspended fourth (A-D-E)	Deliberately unresolved: neither major nor minor.

16. Game-to-music / Strudel system [STRUDEL]

The Strudel subsystem compiles a finished deterministic game record into executable generative music. It is not the adaptive in-game soundtrack. The compiler reconstructs game state turn by turn and maps geometry, move semantics, score, threats, and final state into temporal patterns.

Compiler pipeline

game record -> replay states -> per-turn musical features -> Strudel source template -> encoded Strudel URL / source artifact

Board algorithm

Treat one turn as one bar and the five board columns as five temporal steps. Board row determines pitch within a D-minor-ninth collection (D-F-A-C-E). Squares become plucked low strings, circles soft piano, triads a rising celesta figure through the cleared cells, and hexes bowed strings held from their column to the end of the bar. Persistent hexes therefore accumulate into a growing string chord.

Duet algorithm

Treat each turn as a melodic phrase on a D-Dorian pitch lattice. Moving one cell right advances one scale degree; moving one cell up advances approximately a third. Blue and Red use contrasting instruments and registers. A square is a long low note; a circle is a two-note dependency; a triangle traverses square-circle-triangle and lands on an octave-displaced hex; pass is silence. The non-moving player may quietly echo their previous phrase, creating counterpoint.

Strudel command vocabulary

Expression	Meaning in this specification
setcps(0.45)	Set cycles per second; here approximately one bar/cycle every 2.22 s.
note(pattern)	Interpret numeric/MIDI pattern values as notes.
stack(a,b,c)	Play multiple patterns in parallel.

.s('piano')	Select sound/sample/instrument.
.gain(0.6)	Scale amplitude.
.attack(0.25) / .release(1.2)	Set onset and tail articulation.
.lpf(2200)	Apply a low-pass filter.
.room(0.55).roomsize(6)	Apply room/reverb parameters.
.delay(0.3).delaytime(0.5)	Send to a delay effect and set delay timing.
.pan(0.3)	Place signal in the stereo field.
.velocity(perlin.range(...))	Deterministic time-varying loudness humanization.
.nudge(rand.range(0,0.02))	Small deterministic timing displacement.
~	Rest / silence in mini-notation.
[57,64]	Simultaneous notes / chord.
*8	Repeat event eight times within its pattern span.
@5	Weight/extend event duration by five units.

Annotated example

```

hall(
  note(perBar(t => t.live && t.threat ? '[57,64]' : '~'))
  .s('gm_tremolo_strings')
  .attack(0.6).release(0.8).gain(0.3)
)

```

Interpretation: perBar derives one game-state feature frame for the current turn; the threat predicate gates either a two-note tension sonority or silence; note creates a pitch pattern; s selects tremolo strings; attack/release shape articulation; gain sets layer level; hall is a project helper that adds the shared room treatment.

Deterministic humanization

Do not call uncontrolled random functions while compiling the score. Apparent irregularity may use Strudel's time-indexed noise/random patterns so a given game record produces the same performance on every replay.

17. Online multiplayer architecture [NET]

Online play uses an authoritative server. Clients submit commands; the server validates and applies them using the shared domain package, resolves triangle chance, and publishes the resulting state/version to both clients.

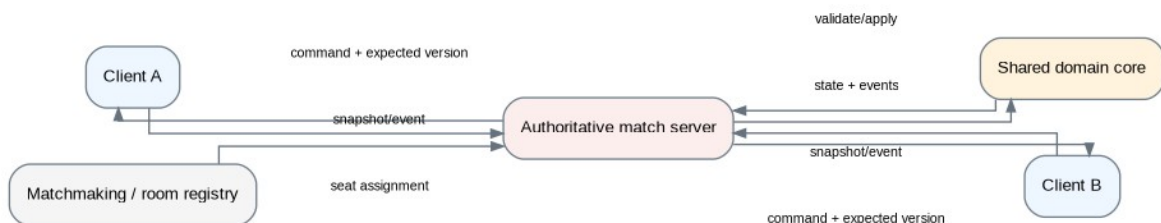


Exhibit 9 — Multiplayer authority boundary — Clients are optimistic only for local presentation effects that cannot affect rules. Canonical state advances only after server acceptance.

Protocol rules

- Every command includes matchId, player seat, client command ID, expected state version, and action.
- The server rejects commands from the wrong seat, wrong turn, stale version, or invalid action.
- Accepted commands increment the match version exactly once and emit an idempotent event.
- Chance resolution occurs only on the server and is included in the response event.
- Clients can recover by requesting a full snapshot plus the latest version.

Matchmaking

Anonymous quick-match queues are partitioned by board size and compatible protocol version. The first client waits; the second fills the room. Seat colors and first player are separate concepts: seat assignment may be deterministic while the first mover is randomly chosen.

Storage and scaling

Define a MatchRepository interface. Development may use in-memory storage; production should support Redis or a durable store so process restarts do not silently destroy active matches. Match state is compact enough to snapshot on every accepted command.

18. Persistence and data contracts [DATA]

Local settings

Persist a single versioned settings object. Validate it through Zod on load and migrate older schema versions explicitly. Never assume localStorage contains trusted data.

```
interface SavedSettingsV1 {
  version: 1; boardSize: 5 | 7; opponent: 'human' | 'ne0n' | 'xen0n';
  mode: 'normal' | 'turbo'; humanMode: 'online' | 'offline'; theme: ThemeId;
  hints: boolean; messages: boolean; sound: boolean; music: boolean; voice: boolean;
}
```

Replay file

A portable replay contains format/version, game configuration, players/agent descriptors, seed/first-player metadata, ordered events, terminal result, optional timestamps, and an integrity hash. Import validates schema before replay.

Model artifact

Neural model metadata includes format ID, model version, board compatibility, input-plane definition, action encoding version, tensor shapes/dtypes, training generation, evaluation summary, and optional golden inference tests.

19. Performance, security and observability [OPS]

Performance budgets

Area	Budget / rule
UI interaction	No synchronous move handler should perform expensive search or model inference.

Rendering	Board updates should fit comfortably inside one frame on mid-range mobile hardware; memoize SVG pieces and derived selectors.
Analysis	Run in Web Worker; yield reports after bounded slices.
Neural inference	Batch where training/self-play benefits; in product, prioritize latency and cancellation over throughput.
Audio	Schedule 200-300 ms ahead while waking roughly every 50 ms; avoid creating unnecessary live noise nodes.
Network	Commands are small; state snapshots are tiny. Prefer correctness/versioning over premature delta compression.

Security boundaries

- Treat all client commands, replay imports, settings, LLM responses, and model files as untrusted input until schema-validated.
- Server secrets and provider API keys never ship in the web bundle.
- Use a restrictive Content-Security-Policy and pinned dependencies.
- Rate-limit matchmaking and command endpoints; cap request bodies and transcript sizes.
- Do not expose other players' identifiers; anonymous play needs only ephemeral match/seat credentials.
- Keep LLM transcripts opt-in or evaluation-only if they may contain provider-generated text.

Observability

Structured logs use correlation IDs for session, match, command, worker request, and network request. Important diagnostics include state hash, state version, action, legality result, chance outcome, agent latency, model version, search depth/nodes, reconnect count, and audio capability—not raw private user content.

20. Accessibility and inclusive design [A11Y]

Accessibility is a product requirement, not a post-processing pass. The game must be operable without sight, without color perception, without precise pointing, and with reduced motion where technically feasible.

Area	Requirement
Keyboard	Complete operation of settings, board, overlays, and result actions.
Screen reader	Board announced as grid; each cell label contains coordinate, occupant, owner, legality, and selected-action relevance.
Color	Ownership and hints have shape/outline/text alternatives; contrast meets WCAG AA.
Motion	All nonessential motion reduced or removed under prefers-reduced-motion.
Touch	Large targets, sufficient spacing, no hover-only actions.
Audio	Important information never exists only as sound.
Voice	Optional enhancement with visible state, clear permission flow, and manual equivalent.
Text	Plain-language tutorial; theme nouns remain short and distinct.

21. Testing strategy [TEST]

The strongest tests are cross-layer invariants that make it difficult for another subsystem to drift away from the game definition.

Required test layers

Layer	Examples
Unit	neighbors, triadPairs, inventory, group scoring, terminal result, action encoding.
Property	Replay round trip; symmetry preserves legality/result; hexCount equals board; no action creates invalid ownership.
Golden vectors	Known states with exact legal actions, chance pairs, transitions, and results.
Differential	Optimized search representation produces the same transitions as game-core on thousands of random states.
Neural parity	Browser forward pass matches training framework within tolerance on exported test vectors.
Component	Board semantics, focus, legal-cell highlighting, pending-hex mode, settings validation.
E2E	New game, tutorial, bot match, offline human, online reconnect, PWA offline start.
Accessibility	axe checks plus manual keyboard and screen-reader scenarios.
Performance	Worker cancellation, inference latency, no long tasks during ordinary move rendering.

Golden assertions

```

ASSERT replay(record).stateHash == record.finalStateHash
ASSERT legalActions(symmetry(s,k)) == symmetry(legalActions(s),k)
ASSERT computeResult(s) == solverTerminalResult(encodeSolverState(s))
ASSERT browserNet(test.obs) ~= trainingNet(test.obs)
ASSERT every agent action ∈ legalActions(state)

```

22. Build, packaging and deployment [BUILD]

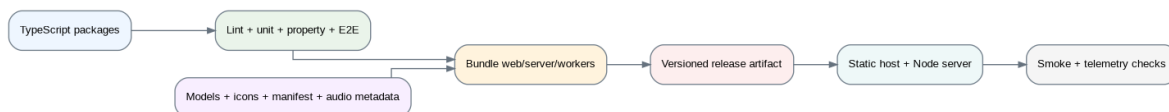


Exhibit 10 — Release pipeline — Source, models, and static assets converge into a versioned release only after deterministic tests and browser-level validation pass.

Repository and toolchain

Use a pnpm workspace with a single lockfile. TypeScript runs in strict mode. Vite builds the browser app and workers. The Node server is compiled separately. CI runs formatting, linting, type checks, unit/property tests, model parity tests, browser tests, and artifact smoke tests before publishing immutable versioned assets.

PWA

- Manifest declares name, icons, theme/background colors, display mode, and preferred portrait orientation without making orientation a rules dependency.
- Service worker uses an explicit cache version and a predictable update strategy; never serve mismatched JS and model artifacts.
- Core local play, tutorial, bots, settings, and audio assets remain available offline after first successful install.
- Online matchmaking and external LLM providers naturally require connectivity.

Release metadata

Expose build ID, git revision, protocol version, game-rule version, model version, and replay format version through a diagnostics panel or machine-readable endpoint. This makes bug reports and multiplayer compatibility decisions actionable.

23. Recommended package tree and dependency policy [PKG]

```
packages/
game-core/      # canonical rules, state, replay, action encoding
game-geometry/  # precomputed neighbors, symmetries, coordinate helpers
game-protocol/  # Zod schemas for commands, snapshots, replays
search/         # expectiminimax, TT, symmetry, analysis reports
neural-runtime/ # browser inference + model loader
agents/         # heuristic, neural, search, LLM, remote adapters
audio-engine/   # Web Audio scheduling, SFX, DSP
music-engine/   # adaptive soundtrack state -> notes
strudel-export/ # replay -> Strudel source
ui-components/  # board, pieces, dialogs, status panels
apps/
web/            # React application
server/         # matchmaking + authoritative matches
tools/
training/       # self-play and model training
benchmarks/     # tournaments, LLM matches, statistics
test-vectors/   # golden states, model parity fixtures
```

Third-party dependency policy

Package	Use	Policy
react / react-dom	Web UI	Pinned major; no domain dependency.
vite	Build/dev/worker bundling	Build-time only.
tailwindcss	Layout and theme utilities	Keep semantic component classes for game-specific states.
xstate	Explicit async UI/application workflows	Use for orchestration, not domain rules.
zod	Runtime schema validation	Network, storage, replay, model metadata.
fastify	HTTP server	Thin transport around MatchService.
ws or socket.io	Realtime match transport	Protocol adapter only.
vitest / fast-check	Unit and property testing	Core packages.
playwright	Cross-browser E2E and PWA tests	CI.
eslint / prettier	Static quality	CI blocking.

--	--	--

24. Configuration registry [CFG]

All behavioral constants live in typed configuration modules. Values below are the prescribed baseline; experiments may override them through explicit configs, never by scattering magic numbers through code.

Key	Default	Scope
board.sizes	[5,7]	Game
board.default	5	Product
squares.initial	round($S^2/3$)	Game
firstPlayer	uniform random	Match
actionEncoding.version	1	AI/protocol
agent.mode.normal.temperature.ne0n	0.65	Opponent
agent.mode.turbo.temperature.ne0n	0.20	Opponent
agent.mode.normal.temperature.xen0n	0.975	Opponent
agent.mode.turbo.temperature.xen0n	0.30	Opponent
rl.search.simulations	32	Training
rl.replay.capacity	250000 decisions	Training
rl.iteration.selfPlay	~8192 decisions	Training
rl.iteration.gradientSteps	~64	Training
rl.promotion.minScore	0.55	Training
audio.scheduler.tick	50 ms	Audio
audio.scheduler.lookahead	250 ms	Audio
audio.echo.feedback	0.42	Audio
audio.echo.lowpass	2400 Hz	Audio
audio.reverb.duration	3.2 s	Audio
online.joinTimeout	20 s	Network
online.healthCheck	20 s	Network

25. Engineering standards and development workflow [ENG]

Coding standards

- TypeScript strict mode; exhaustive switch statements over domain unions.
- Use branded or narrow domain types for PlayerId, CellIndex, MatchId, StateVersion where practical.
- Pure functions for rules and selectors; isolate I/O behind interfaces.
- No catch-all error swallowing. Convert external failures into typed application errors with user-safe messages.
- Every asynchronous API accepts AbortSignal or equivalent cancellation.
- No hidden global mutable state for match, model, or audio session ownership.
- Prefer small modules with explicit public exports over one giant application component.
- Architectural decisions that violate dependency rules require an ADR.

Implementation workflow

7. Create game-core with rules, schemas, invariants, and unit/property tests.
8. Add golden replay vectors and deterministic chance-resolution fixtures.
9. Build a minimal semantic board UI driven directly by immutable GameState.
10. Add MatchController and local human-vs-human play.
11. Add theme renderer, tutorial, accessibility semantics, and persisted settings.
12. Implement heuristic/neural PlayerAgent adapters and worker lifecycle.
13. Implement search/analysis engine and cross-check against game-core.
14. Add audio and generative soundtrack behind event/state adapters.
15. Add authoritative online server and reconnect protocol.
16. Build training/self-play pipeline against the exact same game contracts.
17. Add benchmark/LLM adapters and reproducible result format.
18. Add Strudel compiler and deterministic media artifacts.
19. Run performance, accessibility, security, offline/PWA, and browser compatibility passes.
20. Freeze a release only after the acceptance matrix in Section 26 passes.

26. Acceptance specification [ACCEPT]

Area	Release gate
Rules	All legal/illegal move and terminal golden vectors pass on 5×5 and 7×7.
Replay	Every shipped example replay reconstructs the exact terminal state and result.
GUI	No clipped controls at 320 px width; board remains usable across phone/tablet/desktop target matrix.
Accessibility	Complete keyboard path, screen-reader board semantics, reduced motion, WCAG AA contrast.
Agents	Every returned action is validated; cancellation/stale-result tests pass.
Neural	Browser parity test within agreed numeric tolerance for every export.
Search	Differential random-state tests match game-core; exact mode labels only exact results.
Online	Authoritative stale-version rejection, reconnect snapshot, duplicate-command idempotency, chance consistency.
Audio	Game playable with audio unavailable; no audible clipping; hidden/suspended tab behaves predictably.
Offline	Installed/previously loaded app starts and supports local play without network.
Security	No secrets in client bundle; schema validation and rate limits active; dependency audit reviewed.
Build	Artifact contains version manifest and passes clean-install smoke test.

27. Reference ontology and glossary [GLOSSARY]

Term	Canonical definition
------	----------------------

Action	A player proposal such as placing one piece or passing.
Decision	A point at which a player has more than zero choices. A triangle and its following hex are separate decisions by the same player.
Turn	A completed player turn; a triangle plus its mandatory hex still belongs to one turn.
Chance node	A stochastic branch controlled by the game, not by either player.
Stable state	A state between completed turns with no pending hex.
Pending-hex state	Transient state after triangle resolution in which the same player must place a hex.
Canonical state	The authoritative state representation exposed by game-core.
Transposition	The same game state reached by different move sequences.
Canonicalization	Mapping symmetry-equivalent states to one representative key.
Policy	Probability distribution or scores over legal actions.
Value	Expected game outcome, conventionally in [-1,+1].
Intensity	Music-control scalar derived from game progress/urgency.
Pattern	A Strudel time-varying musical value, not a one-shot imperative event.
Adapter	Implementation of a port that talks to an external subsystem or specialized runtime.
State hash	Deterministic digest used to identify a canonical state and reject stale async results.

A. Appendix A — Formal rule predicates [APP-A]

```

empty(i) := board[i].piece == EMPTY
canSquare(p,i) := empty(i) && squaresLeft[p] > 0
canCircle(i) := empty(i) && ∃j∈N(i): piece(j)=SQUARE
canTriangle(p,i) := empty(i) && |Pairs(i,p)| > 0
hasLegalPlacement(p) := ∃i: canSquare(p,i) ∨ canCircle(i) ∨ canTriangle(p,i)
canPass(p) := pendingHex=null && !hasLegalPlacement(p)

```

B. Appendix B — Example replay notation [APP-B]

For compact human-readable artifacts, use tokens separated by spaces. The machine format remains structured JSON events.

```

S<cell>
C<cell>
T<triangle>-<circle>-<square>-<hex>
P

```

Example: `S17 S14 C16 C11 T12-16-17-16 ...` alternates players beginning with the replay's firstPlayer metadata. The triad token includes the resolved chance pair and chosen hex destination, making it self-contained.

C. Appendix C — Example protocol messages [APP-C]

```
{
  "type": "command",
  "matchId": "m_123",
  "commandId": "c_007",
  "expectedVersion": 12,
  "action": {
    "kind": "PLACE_CIRCLE",
    "cell": 8
  }
}

{
  "type": "accepted",
  "matchId": "m_123",
  "version": 13,
  "events": [...],
  "state": {...}
}
```

D. Appendix D — Architecture adjacency list [APP-D]

```
web-ui -> match-controller
match-controller -> game-core
match-controller -> PlayerAgent
match-controller -> MatchTransport
match-controller -> AudioPort
match-controller -> PersistencePort
server -> game-core
search -> game-core test vectors
neural-runtime -> model artifact
training -> game-core + search + neural framework
strudel-export -> replay + derived musical features
```

E. Appendix E — Definition of done for a new subsystem [APP-E]

21. Public contract written and versioned.
22. Dependency direction conforms to Section 5.
23. Inputs are runtime-validated at external boundaries.
24. Cancellation and stale-result behavior defined if asynchronous.
25. Unit tests and at least one integration/golden test exist.
26. Failure behavior degrades without corrupting canonical game state.
27. Observability fields are defined.
28. Accessibility impact is reviewed if user-facing.
29. Configuration constants are registered, not embedded ad hoc.
30. Documentation contains one worked example.

Implementation north star

The product may gain new themes, agents, music mappings, analysis tools, and transports without changing the semantic center: one canonical game state, one rules engine, one replay model, and explicit typed boundaries around everything else.